Image Filtering — Exam Notes (Computer Vision)

> Compact, exam-oriented notes for B.Tech CSE semester exams & quick revision.

Image Filtering

Definition

- **Image filtering** is the process of modifying pixel values using a mathematical operation to enhance, suppress, or extract specific features of an image.
- It is a core **pre-processing** step in Computer Vision.

Why is it used?

- To remove **noise**
- To **enhance** edges/features
- To **smooth** or **sharpen** images
- To prepare images for **segmentation, detection, recognition**

How does it work?

- A small window (**kernel**) slides over the image.
- At each pixel, a mathematical operation (sum, average, weighted sum) combines the neighborhood pixel values.
- The result replaces the center pixel value in the output image.
- Filtering can be done in **spatial domain** (direct pixel operations) or **frequency domain** (after Fourier Transform).

Formula / Kernel (if applicable)

- General linear filtering:

```
g(x,y) = Σ Σ f(x+i, y+j) * h(i,j)
```

where `f` = input image, `h` = kernel/mask, `g` = output image.

Advantages

- Improves image quality
- Enables feature extraction
- Reduces noise before further processing

Disadvantages

- May blur important details
- Choice of kernel size affects result quality
- Computationally expensive for large kernels

Applications

- Medical imaging, satellite imaging, face recognition, OCR, object detection

Exam Points

- Filtering = **neighborhood processing**, unlike point processing (single pixel operation).
- Two domains: **Spatial** and **Frequency**.
- Filtering can be **linear** (mean, Gaussian) or **non-linear** (median).

Python OpenCV Example

```python
import cv2
img = cv2.imread('image.jpg', 0)          # read grayscale
filtered = cv2.blur(img, (5,5))           # simple averaging filter
cv2.imwrite('filtered.jpg', filtered)
```

Filter, Kernel, Mask

Definition

- **Filter**: A general operation applied to an image to modify it (e.g., smoothing, sharpening).
- **Kernel**: A small matrix of numbers used to perform the filtering operation via convolution.
- **Mask**: Often used interchangeably with kernel; sometimes refers to a binary matrix used to select a region of interest.

Why is it used?

- Kernel/mask defines **how much weight** each neighboring pixel contributes to the output pixel.

How does it work?

- The kernel is placed over each pixel neighborhood.
- Element-wise multiplication is performed between kernel values and pixel values.
- The products are summed to get the new pixel value.
- Kernel size is usually **odd** (3x3, 5x5) so there is a clear center pixel.

Formula / Kernel (if applicable)

Example 3x3 kernel (identity — no change):

```
0 0 0
0 1 0
0 0 0
```

Advantages

- Simple and flexible way to represent many operations (blur, sharpen, edge detect) using different kernel values.

Disadvantages

- Larger kernels → higher computation cost.
- Wrong kernel choice can distort the image.

Applications

- Used in blurring, sharpening, edge detection, embossing, etc.

Exam Points

- **Kernel = Filter = Mask** (mostly synonymous in exams, minor contextual difference).
- Kernel size must be **odd** for a defined center.
- Sum of kernel values = 1 → preserves brightness (used in smoothing filters).

Python OpenCV Example

```python
import cv2, numpy as np
img = cv2.imread('image.jpg', 0)
kernel = np.array([[0,-1,0],
                   [-1,5,-1],
                   [0,-1,0]])              # sharpening kernel
output = cv2.filter2D(img, -1, kernel)
cv2.imwrite('sharpened.jpg', output)
```

Convolution

Definition

- **Convolution** is a mathematical operation that combines an image with a kernel to produce a filtered output by flipping the kernel and sliding it across the image.

Why is it used?

- Forms the **mathematical basis** of most spatial filtering operations (smoothing, sharpening, edge detection).
- Basis of **CNNs** in deep learning.

How does it work?

- The kernel is **flipped (rotated 180°)** before sliding (unlike correlation, where it is not flipped).
- For each position, overlapping values are multiplied and summed.
- The kernel moves across the entire image, producing the output image.
- **Padding** is used to handle border pixels (zero padding, replicate, reflect).

Formula / Kernel (if applicable)

```
g(x,y) = Σ_m Σ_n f(x-m, y-n) * h(m,n)
```

- `*` denotes convolution.
- Note: **Correlation** skips the flipping step: `g(x,y) = ΣΣ f(x+m,y+n)*h(m,n)`

Advantages

- Mathematically well-defined, supports frequency-domain equivalence (Convolution Theorem).
- Efficient for extracting local features.

Disadvantages

- Border pixels need special handling (padding).
- Computationally expensive for large images/kernels (solved using FFT-based convolution).

Applications

- Blurring, sharpening, edge detection, CNN feature extraction.

Exam Points

- **Convolution Theorem**: Convolution in spatial domain = Multiplication in frequency domain.
- For **symmetric kernels** (like Gaussian), convolution = correlation.
- OpenCV's `filter2D()` actually performs **correlation**, not true convolution (kernel not flipped).

Python OpenCV Example

```python
import cv2, numpy as np
img = cv2.imread('image.jpg', 0)
kernel = np.ones((3,3), np.float32) / 9      # averaging kernel
conv_result = cv2.filter2D(img, -1, kernel)
cv2.imwrite('conv_output.jpg', conv_result)
```

Types of Filtering (Spatial & Frequency Domain)

Definition

- **Spatial domain filtering**: Operates directly on pixel values of the image.
- **Frequency domain filtering**: Operates on the image after transforming it (usually via **Fourier Transform**) into frequency components.

Why is it used?

- Spatial: simple, intuitive, good for local features (edges, noise).
- Frequency: good for global patterns, periodic noise removal.

How does it work?

- **Spatial**: kernel convolved directly with pixel neighborhoods.
- **Frequency**: Image → **FFT** → multiply with filter mask (low-pass/high-pass) → **Inverse FFT** → filtered image.

Formula / Kernel (if applicable)

```
Spatial:    g(x,y) = f(x,y) * h(x,y)
Frequency:  G(u,v) = F(u,v) . H(u,v)
```

Advantages

Domain	Advantage
Spatial	Simple, fast for small kernels, intuitive
Frequency	Efficient for large-scale/periodic filtering, easy to design ideal filters

Disadvantages

Domain	Disadvantage
Spatial	Slow for large kernels
Frequency	Requires FFT computation, ringing artifacts (Gibbs phenomenon)

Applications

- Spatial: smoothing, sharpening, edge detection
- Frequency: removing periodic noise, image compression, texture analysis

Exam Points

- **Convolution Theorem** connects the two domains.
- Frequency domain filters: **Low-pass** (blurring), **High-pass** (edge enhancement), **Band-pass/Band-reject**.
- FFT-based filtering is efficient for **large kernels**.

Python OpenCV Example

```python
```

```python
import cv2, numpy as np
img = cv2.imread('image.jpg', 0)
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)
magnitude = 20*np.log(np.abs(fshift)+1)
cv2.imwrite('freq_spectrum.jpg', magnitude.astype(np.uint8))
```

Smoothing Filters

Definition

- **Smoothing (blurring) filters** reduce noise and detail by averaging pixel values in a neighborhood, resulting in a softer image.

Why is it used?

- Noise reduction before further processing
- Removing small unwanted details
- Pre-processing for edge detection (reduces false edges)

How does it work?

- Each output pixel is a **weighted or unweighted average** of its neighbors.
- Larger kernel → stronger blurring effect.
- Types: **Mean, Gaussian, Median** (median is non-linear).

Formula / Kernel (if applicable)

- General smoothing: `g(x,y) = average of f in neighborhood of (x,y)`

Advantages

- Reduces noise, prepares image for further analysis

Disadvantages

- Blurs edges and fine details
- Over-smoothing loses important information

Applications

- Pre-processing, denoising, background blurring

Exam Points

- Smoothing = **Low-pass filtering**.
- Larger kernel size = more blur, less detail.
- Trade-off between **noise removal** and **detail preservation**.

Python OpenCV Example

```python
```

```
import cv2
img = cv2.imread('image.jpg', 0)
smooth = cv2.GaussianBlur(img, (5,5), 0)
cv2.imwrite('smooth.jpg', smooth)
```

Mean Filter

Definition

- The **Mean (Averaging) Filter** replaces each pixel with the **average** of pixel values in its neighborhood.

Why is it used?

- Simple, fast noise reduction (especially Gaussian noise)

How does it work?

- Slide an **n×n kernel** (all values equal) over the image.
- Replace center pixel with the **mean** of the neighborhood pixels.
- Larger kernel = stronger blur.

Formula / Kernel (if applicable)

3x3 Mean kernel:

```
1/9 * | 1 1 1 |
      | 1 1 1 |
      | 1 1 1 |
```

Advantages

- Simple, fast, easy to implement
- Effective for **Gaussian noise**

Disadvantages

- Blurs edges significantly
- Does not preserve sharp details
- Ineffective against **salt-and-pepper noise**

Applications

- Basic noise smoothing, pre-processing step

Exam Points

- Also called **Box filter** / **Averaging filter**.
- All kernel weights are **equal.**
- **Linear filter.**

Python OpenCV Example

```python
import cv2
img = cv2.imread('image.jpg', 0)
mean_filtered = cv2.blur(img, (5,5))
cv2.imwrite('mean_filtered.jpg', mean_filtered)
```

Gaussian Filter

Definition

- The **Gaussian Filter** smooths an image using weights derived from the **Gaussian (normal) distribution**, giving more weight to the center pixel.

Why is it used?

- Effective, natural-looking blur
- Reduces noise while preserving edges better than mean filter

How does it work?

- Kernel weights follow a **bell curve**: center pixel gets highest weight, decreasing outward.
- Weighted sum of neighborhood pixels replaces the center pixel.
- Controlled by **standard deviation (σ)** — larger σ = more blur.

Formula / Kernel (if applicable)

2D Gaussian function:

```
G(x,y) = (1 / (2πσ²)) * e^(-(x²+y²) / (2σ²))
```

Example 3x3 approx. kernel (σ small):

```
1/16 * | 1 2 1 |
       | 2 4 2 |
       | 1 2 1 |
```

Advantages

- Smooths noise while preserving more structure than mean filter
- Rotationally symmetric, no directional bias

Disadvantages

- Still blurs edges (less than mean, but present)
- Choosing σ and kernel size needs care

Applications

- Noise reduction, pre-processing for **Canny edge detection**, image pyramids

Exam Points

- **Linear** and **separable** filter (can be applied as 1D horizontal + 1D vertical pass — faster).
- Weight depends on **distance from center** (Gaussian distribution).
- Used as **first step in Canny Edge Detection**.

Python OpenCV Example

```python
import cv2
img = cv2.imread('image.jpg', 0)
gaussian = cv2.GaussianBlur(img, (5,5), sigmaX=1.5)
cv2.imwrite('gaussian.jpg', gaussian)
```

Median Filter

Definition

- The **Median Filter** replaces each pixel with the **median** value of its neighborhood — a **non-linear** filter.

Why is it used?

- Excellent for removing **salt-and-pepper noise** while preserving edges

How does it work?

- Neighborhood pixel values are **sorted**.
- The **middle (median) value** replaces the center pixel.
- Extreme values (noise spikes) get eliminated since they rarely become the median.

Formula / Kernel (if applicable)

- No fixed kernel weights; operation = `sort neighborhood values → pick median`.

Advantages

- Preserves edges better than mean/Gaussian
- Very effective against **impulse (salt-and-pepper) noise**

Disadvantages

- Slower than mean/Gaussian (requires sorting)
- Not effective against Gaussian noise
- Can remove small fine details/thin lines

Applications

- Noise removal in scanned documents, medical images, salt-and-pepper noise cleanup

Exam Points

- **Non-linear filter** (unlike mean & Gaussian).
- Best filter for **salt-and-pepper noise**.
- Preserves **edges** while removing noise.

Python OpenCV Example

```python
import cv2
img = cv2.imread('image.jpg', 0)
median = cv2.medianBlur(img, 5)          # kernel size must be odd
cv2.imwrite('median.jpg', median)
```

Sharpening Filters

Definition

- **Sharpening filters** enhance edges and fine details by **highlighting intensity discontinuities** (high-frequency components) in an image.

Why is it used?

- Improve visual clarity of edges/details
- Counteract blurring introduced during acquisition/transmission

How does it work?

- Emphasizes differences between neighboring pixels.
- Achieved using **derivative-based** kernels (Laplacian) or **unsharp masking**.
- Adds high-frequency information back to the image.

Formula / Kernel (if applicable)

- General idea: `Sharpened = Original + k * (High-frequency component)`

Advantages

- Enhances edges, improves perceived quality

Disadvantages

- Amplifies noise along with edges
- Can create ringing/overshoot artifacts

Applications

- Medical imaging, satellite image enhancement, photography

Exam Points

- Sharpening = **High-pass filtering.**
- Opposite effect of smoothing filters.
- Common methods: **Laplacian filter, Unsharp Masking, High-boost filtering.**

Python OpenCV Example

```python
import cv2, numpy as np
img = cv2.imread('image.jpg', 0)
kernel = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
sharp = cv2.filter2D(img, -1, kernel)
cv2.imwrite('sharp.jpg', sharp)
```

Laplacian Filter

Definition

- The **Laplacian filter** is a **second-order derivative** operator used to detect edges by highlighting regions of rapid intensity change, regardless of direction.

Why is it used?

- Edge detection
- Image sharpening (add Laplacian back to original)

How does it work?

- Computes the **second derivative** of image intensity.
- Zero-crossings in the Laplacian indicate edges.
- Since it's **isotropic** (rotation-invariant), it detects edges in **all directions** simultaneously.
- Very sensitive to noise → often preceded by Gaussian smoothing (→ **LoG: Laplacian of Gaussian**).

Formula / Kernel (if applicable)

$$\nabla^2 f = \partial^2 f/\partial x^2 + \partial^2 f/\partial y^2$$

Common 3x3 kernels:

```
 0  1  0        -1 -1 -1
 1 -4  1   or   -1  8 -1
 0  1  0        -1 -1 -1
```

Advantages

- Detects edges in **all directions** (isotropic)
- Simple, single-pass operation

Disadvantages

- Highly **sensitive to noise**
- Produces **double edges** (positive & negative response)
- Does not give edge direction (unlike Sobel)

Applications

- Edge detection, image sharpening, blob/feature detection (LoG, DoG in SIFT)

Exam Points

- **Second-order derivative** filter (Sobel is first-order).
- Detects edges via **zero-crossing**.
- **LoG (Laplacian of Gaussian)** = Gaussian smoothing + Laplacian → reduces noise sensitivity.
- Sum of kernel values = **0**.

Python OpenCV Example

```python
import cv2
img = cv2.imread('image.jpg', 0)
lap = cv2.Laplacian(img, cv2.CV_64F, ksize=3)
lap_abs = cv2.convertScaleAbs(lap)
cv2.imwrite('laplacian.jpg', lap_abs)
```

Unsharp Masking

Definition

- **Unsharp Masking** sharpens an image by subtracting a **blurred (unsharp) version** from the original and adding the resulting high-frequency detail back.

Why is it used?

- Effective, controllable sharpening technique used in photography and scanning

How does it work?

1. Create a **blurred copy** of the original image (e.g., Gaussian blur).
2. Subtract blurred image from original → **mask** (edge/detail information).
3. Add a scaled version of the mask back to the original image.

Formula / Kernel (if applicable)

```
Sharpened = Original + k * (Original - Blurred)
```

where `k` = sharpening strength (amount).

Advantages

- Adjustable strength (via `k` and blur radius)
- Produces natural-looking sharpening

Disadvantages

- Can amplify noise
- Overuse causes **halo artifacts** near edges

Applications

- Photo editing software, medical/satellite image enhancement, printing/scanning

Exam Points

- Formula: `Sharpened = Original + k(Original - Blurred)`.
- Also called **High-boost filtering** when `k > 1`.
- Requires a **blurring step first** (usually Gaussian).

Python OpenCV Example

```python
import cv2
img = cv2.imread('image.jpg', 0)
blurred = cv2.GaussianBlur(img, (9,9), 10)
sharpened = cv2.addWeighted(img, 1.5, blurred, -0.5, 0)
cv2.imwrite('unsharp.jpg', sharpened)
```

Edge Detection

Definition

- **Edge detection** identifies points in an image where **intensity changes sharply**, marking boundaries of objects.

Why is it used?

- Object boundary detection
- Feature extraction for recognition/segmentation

How does it work?

- Uses **derivative operators** (first or second order) to detect intensity discontinuities.
- **First-order** (gradient-based): Sobel, Prewitt, Roberts.

- **Second-order** (zero-crossing based): Laplacian, LoG.
- **Canny** combines gradient + non-max suppression + hysteresis thresholding for optimal results.

Formula / Kernel (if applicable)

```
Gradient magnitude: |∇f| = √(Gx² + Gy²)
Gradient direction:  θ = tan⁻¹(Gy / Gx)
```

Advantages

- Reduces data while preserving structural information
- Basis for object recognition, segmentation

Disadvantages

- Sensitive to noise
- Thresholds must be chosen carefully

Applications

- Object detection, medical image analysis, lane detection (self-driving cars), OCR

Exam Points

- Types: **First-order (gradient)** and **Second-order (Laplacian)**.
- **Canny** = most accurate & widely used edge detector.
- Edge strength = **gradient magnitude**; edge direction = **gradient angle**.

Python OpenCV Example

```python
import cv2
img = cv2.imread('image.jpg', 0)
edges = cv2.Canny(img, 100, 200)
cv2.imwrite('edges.jpg', edges)
```

Sobel Filter

Definition

- The **Sobel filter** is a **first-order derivative** edge detector that computes the **gradient** of image intensity in horizontal and vertical directions.

Why is it used?

- Detect edges with direction information
- Simple and effective gradient-based edge detection

How does it work?

- Two kernels: **Gx** (detects vertical edges) and **Gy** (detects horizontal edges).
- Convolve image with both kernels separately.
- Combine using gradient magnitude formula.
- Includes a built-in **smoothing effect** (less noise-sensitive than simple derivative).

Formula / Kernel (if applicable)

```
Gx =  -1 0 1        Gy =  -1 -2 -1
      -2 0 2               0  0  0
      -1 0 1               1  2  1

Magnitude: G = √(Gx² + Gy²)
Direction: θ = tan⁻¹(Gy/Gx)
```

Advantages

- Provides both **magnitude & direction** of edges
- Some noise resistance due to smoothing component

Disadvantages

- Still sensitive to noise (less than raw gradient)
- Produces **thick edges** compared to Canny

Applications

- Edge detection pre-processing, feature extraction, object boundary detection

Exam Points

- **First-order derivative** operator.
- Two separate kernels for **x** and **y** direction.
- Combines **smoothing + differentiation** in one kernel (unlike Roberts/Prewitt which are purely differential).

Python OpenCV Example

```python
import cv2
img = cv2.imread('image.jpg', 0)
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sobelx, sobely)
cv2.imwrite('sobel.jpg', cv2.convertScaleAbs(sobel))
```

Canny Edge Detector

Definition

- The **Canny Edge Detector** is a **multi-stage, optimal edge detection algorithm** that produces thin, accurate, well-connected edges with minimal noise.

Why is it used?

- Considered the **best general-purpose edge detector**
- Balances noise suppression and accurate edge localization

How does it work? (5 Steps)

1. **Noise reduction** – Gaussian blur applied first.
2. **Gradient calculation** – Sobel operator finds intensity gradient (magnitude + direction).
3. **Non-maximum suppression** – thin edges by keeping only local maxima along gradient direction.
4. **Double thresholding** – classify pixels as **strong, weak, or non-edges** using high & low thresholds.
5. **Edge tracking by hysteresis** – weak edges connected to strong edges are kept; others discarded.

Formula / Kernel (if applicable)

```
High threshold : strong edge
Low threshold  : weak edge  (kept only if connected to strong edge)
Below low threshold: suppressed (not an edge)
```

Advantages

- Produces **thin, accurate** edges
- Good noise immunity due to Gaussian smoothing
- Fewer false edges due to hysteresis thresholding

Disadvantages

- Computationally more expensive (multi-stage)
- Performance depends on **threshold selection**

Applications

- Autonomous vehicles (lane/object detection), medical imaging, industrial inspection, OCR pre-processing

Exam Points

- **5 steps**: Smoothing → Gradient → Non-max suppression → Double thresholding → Hysteresis.
- Uses **Sobel operator internally** for gradient computation.

- Two thresholds: **T_high** and **T_low** (typical ratio 2:1 or 3:1).
- Regarded as the **optimal edge detector** (satisfies good detection, good localization, single response criteria — Canny's 3 criteria).

Python OpenCV Example

```python
import cv2
img = cv2.imread('image.jpg', 0)
edges = cv2.Canny(img, threshold1=100, threshold2=200)
cv2.imwrite('canny_edges.jpg', edges)
```

Noise Reduction

Definition

- **Noise reduction** is the process of removing unwanted random variations (noise) in pixel intensity to recover a cleaner image.

Why is it used?

- Improve image quality
- Improve accuracy of downstream tasks (edge detection, segmentation, recognition)

How does it work?

- Depends on **noise type**:
  - **Gaussian noise** → Gaussian/Mean filter
  - **Salt-and-pepper noise** → Median filter
  - **Periodic noise** → Frequency domain notch/band-reject filter
- General approach: apply appropriate **smoothing filter** matched to the noise characteristics.

Formula / Kernel (if applicable)

- Noisy image model: `g(x,y) = f(x,y) + n(x,y)` where `f` = original image, `n` = noise

Advantages

- Improves signal-to-noise ratio (SNR)
- Enhances visual quality and downstream algorithm accuracy

Disadvantages

- Excessive noise reduction blurs details
- Wrong filter choice may not remove specific noise type effectively

Applications

- Medical imaging (MRI/CT denoising), astrophotography, old photo restoration

Exam Points

- No single filter removes all noise types — **filter choice depends on noise type.**
- **Median filter → salt-and-pepper, Gaussian/Mean → Gaussian noise.**
- Trade-off: **noise removal vs. detail preservation.**

Python OpenCV Example

```python
import cv2
img = cv2.imread('noisy.jpg', 0)
denoised = cv2.fastNlMeansDenoising(img, h=10)   # Non-local means denoising
cv2.imwrite('denoised.jpg', denoised)
```

Frequency Domain Filters

Definition

- **Frequency domain filters** modify an image by operating on its **Fourier Transform**, altering specific frequency components before converting back to spatial domain.

Why is it used?

- Effective removal of **periodic/structured noise**
- Efficient large-scale smoothing/sharpening

How does it work?

1. Convert image to frequency domain using **FFT (Fast Fourier Transform)**.
2. Apply a **filter mask**: Low-pass (blocks high frequencies), High-pass (blocks low frequencies), Band-pass/Band-reject.
3. Multiply Fourier-transformed image with the filter mask.
4. Apply **Inverse FFT** to get filtered image back in spatial domain.

Formula / Kernel (if applicable)

```
G(u,v) = H(u,v) . F(u,v)
g(x,y) = Inverse FFT[G(u,v)]
```

- **Ideal Low-Pass Filter (ILPF):** `H(u,v) = 1 if D(u,v) ≤ D0, else 0`

Advantages

- Efficient for large kernels / global patterns
- Good for removing **periodic noise**

Disadvantages

- **Ringing artifacts** (Gibbs phenomenon) especially with ideal filters
- Requires FFT computation overhead

Applications

- Removing scan-line/periodic noise, image compression (JPEG uses DCT), texture analysis

Exam Points

- **Low-pass filter** → smoothing (removes high-frequency noise/details).
- **High-pass filter** → sharpening (removes low-frequency, keeps edges).
- **Butterworth filter**: smoother transition than Ideal filter (avoids ringing).
- **Gaussian filter in frequency domain**: smoothest transition, no ringing.

Python OpenCV Example

```python
import cv2, numpy as np
img = cv2.imread('image.jpg', 0)
f = np.fft.fftshift(np.fft.fft2(img))
rows, cols = img.shape
mask = np.zeros((rows, cols), np.uint8)
cv2.circle(mask, (cols//2, rows//2), 30, 1, -1)   # low-pass circular mask
filtered = np.fft.ifft2(np.fft.ifftshift(f * mask))
result = np.abs(filtered).astype(np.uint8)
cv2.imwrite('freq_filtered.jpg', result)
```

Morphological Operations

Definition

- **Morphological operations** process images based on **shape**, typically applied to **binary (or grayscale)** images using a **structuring element (kernel)**.

Why is it used?

- Remove noise, fill gaps, separate/join objects
- Extract structural features (boundaries, skeletons)

How does it work?

- A **structuring element** (small binary shape, e.g., 3x3 square/cross) slides over the image.
- Basic operations: **Erosion** (shrinks objects), **Dilation** (grows objects).
- Compound operations: **Opening** (erosion→dilation, removes small noise), **Closing** (dilation→erosion, fills small holes).

Formula / Kernel (if applicable)

```
Opening:  A ∘ B = (A ⊖ B) ⊕ B
Closing:  A • B = (A ⊕ B) ⊖ B
```

($\ominus$ = erosion, $\oplus$ = dilation)

Advantages

- Effective for **binary image cleanup**
- Useful for shape analysis, boundary extraction

Disadvantages

- Choice of structuring element size/shape affects results significantly
- Mainly effective on binary/near-binary images

Applications

- Document/text image cleanup, fingerprint processing, object counting, medical image segmentation

Exam Points

- Two basic ops: **Erosion & Dilation**; compound: **Opening & Closing**.
- **Opening** removes small objects/noise (outside boundary).
- **Closing** fills small holes (inside boundary).
- Requires a **structuring element** (like a kernel, but usually binary: 0s and 1s).

Python OpenCV Example

```python
import cv2, numpy as np
img = cv2.imread('binary.jpg', 0)
kernel = np.ones((5,5), np.uint8)
opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
closing = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)
cv2.imwrite('opening.jpg', opening)
cv2.imwrite('closing.jpg', closing)
```

Erosion

Definition

- **Erosion** shrinks the boundaries of foreground (white) objects in a binary image by removing pixels at object edges.

Why is it used?

- Remove small noise specks
- Shrink/thin objects, separate connected/touching objects

How does it work?

- A structuring element slides over the image.

- Output pixel = **1 (foreground) only if ALL pixels under the kernel are 1**; otherwise set to 0.
- Effectively **erodes away** boundaries of foreground objects.

Formula / Kernel (if applicable)

```
(A ⊖ B)(x,y) = 1  if B fits entirely inside A at (x,y); else 0
```

Advantages

- Removes small noise/protrusions
- Separates weakly connected objects

Disadvantages

- Shrinks useful object regions too
- May completely remove very small objects

Applications

- Noise removal, separating touching objects, thinning shapes

Exam Points

- Shrinks **white regions**, grows **black/background regions**.
- Removes small white noise spots.
- Opposite operation of **Dilation**.

Python OpenCV Example

```python
import cv2, numpy as np
img = cv2.imread('binary.jpg', 0)
kernel = np.ones((3,3), np.uint8)
eroded = cv2.erode(img, kernel, iterations=1)
cv2.imwrite('eroded.jpg', eroded)
```

Dilation

Definition

- **Dilation** expands the boundaries of foreground (white) objects in a binary image by adding pixels at object edges.

Why is it used?

- Fill small holes/gaps in objects
- Join broken/nearby object parts
- Increase object size for better visibility

How does it work?

- A structuring element slides over the image.
- Output pixel = **1 if ANY pixel under the kernel is 1**; otherwise set to 0.
- Effectively **grows** the foreground region.

Formula / Kernel (if applicable)

```
(A ⊕ B)(x,y) = 1  if B overlaps at least one foreground pixel of A at (x,y);
else 0
```

Advantages

- Fills small holes/gaps within objects
- Connects broken parts of an object

Disadvantages

- Can merge nearby distinct objects unintentionally
- Increases object size (may exaggerate noise if applied without erosion)

Applications

- Filling gaps in edges/text, joining broken contours, object boundary thickening

Exam Points

- Grows **white regions**, shrinks **black regions**.
- Fills small holes/gaps.
- Opposite operation of **Erosion**.
- Erosion + Dilation together = **Opening/Closing**.

Python OpenCV Example

```python
import cv2, numpy as np
img = cv2.imread('binary.jpg', 0)
kernel = np.ones((3,3), np.uint8)
dilated = cv2.dilate(img, kernel, iterations=1)
cv2.imwrite('dilated.jpg', dilated)
```

Orthogonal Filters / Transforms (Brief)

Definition

- **Orthogonal transforms** convert an image into a set of **uncorrelated coefficients** using an orthogonal basis, enabling compact representation, compression, and frequency-based filtering.

Why is it used?

- Image compression (energy compaction)
- Feature extraction, frequency analysis
- Efficient filtering in transform domain

How does it work?

- Image is projected onto a set of **orthogonal basis functions.**
- Most image energy gets concentrated into a **few significant coefficients.**
- Common transforms: **DFT (Discrete Fourier Transform), DCT (Discrete Cosine Transform), Walsh-Hadamard Transform, Wavelet Transform.**

Formula / Kernel (if applicable)

- DCT (used in JPEG):

```
F(u,v) = ΣΣ f(x,y) cos[(2x+1)uπ/2N] cos[(2y+1)vπ/2N]
```

Advantages

- Good **energy compaction** (DCT) → efficient compression
- Enables frequency-domain analysis/filtering

Disadvantages

- Computationally intensive for large images
- Some transforms (DFT) need complex number handling

Applications

- **JPEG compression (DCT),** image watermarking, texture/feature analysis, wavelet-based denoising

Exam Points

- **DFT** → complex frequency representation; **DCT** → real-valued, better energy compaction, used in **JPEG.**
- **Walsh-Hadamard** → uses only +1/-1 values, very fast (no multiplication).
- **Wavelet Transform** → provides both **time and frequency** localization (used in JPEG2000).

Python OpenCV Example

python

```python
import cv2, numpy as np
img = cv2.imread('image.jpg', 0).astype(np.float32)
dct = cv2.dct(img)                    # Discrete Cosine Transform
idct = cv2.idct(dct)                  # Inverse DCT (reconstruction)
cv2.imwrite('dct_reconstructed.jpg', idct.astype(np.uint8))
```

Types of Image Noise

Definition

- **Image noise** refers to random, unwanted variations in pixel intensity that degrade image quality, usually introduced during acquisition or transmission.

Why is it studied?

- Choosing the correct **denoising filter** requires knowing the noise type.

How does it work? (Types)

- **Gaussian Noise**: intensity values follow a normal distribution; caused by sensor/electronic noise.
- **Salt-and-Pepper Noise**: random black & white pixels; caused by transmission errors/faulty sensors.
- **Speckle Noise**: multiplicative noise, common in radar/ultrasound images (grainy appearance).
- **Poisson (Shot) Noise**: caused by statistical nature of photon counting, common in low-light imaging.
- **Periodic Noise**: repetitive patterns, caused by electrical/mechanical interference during acquisition.

Formula / Kernel (if applicable)

```
Gaussian noise model: g(x,y) = f(x,y) + n(x,y),  n ~ N(0, σ²)
Salt-and-pepper: pixel randomly set to 0 (pepper) or 255 (salt)
```

Advantages

(N/A — noise is undesirable; understanding it helps choose correct filters)

Disadvantages

- Degrades image quality, reduces accuracy of vision algorithms

Applications (where noise commonly appears)

- Low-light photography, medical ultrasound (speckle), satellite transmission (salt-and-pepper/periodic), old scanned documents

Exam Points

- **Gaussian noise → Gaussian/Mean filter**
- **Salt-and-pepper noise → Median filter**
- **Speckle noise → common in ultrasound/SAR (radar) images**
- **Periodic noise → removed using frequency domain notch filters**

Python OpenCV Example

```
python
```

```python
import cv2, numpy as np
img = cv2.imread('image.jpg', 0)

# Add Gaussian noise
gauss = np.random.normal(0, 25, img.shape).astype(np.float32)
noisy_gaussian = np.clip(img + gauss, 0, 255).astype(np.uint8)

# Add Salt-and-Pepper noise
sp = img.copy()
prob = 0.05
rnd = np.random.rand(*img.shape)
sp[rnd < prob/2] = 0
sp[rnd > 1 - prob/2] = 255

cv2.imwrite('gaussian_noise.jpg', noisy_gaussian)
cv2.imwrite('salt_pepper_noise.jpg', sp)
```

---

1. Comparison Table — All Filters

Filter	Type	Domain	Linear/Non-linear	Best For	Kernel Example
Mean	Smoothing	Spatial	Linear	Gaussian noise	Equal weights (1/9 each)
Gaussian	Smoothing	Spatial	Linear	General noise, pre-Canny	Bell-curve weights
Median	Smoothing	Spatial	Non-linear	Salt-and-pepper noise	No fixed weights (uses sorting)
Laplacian	Sharpening/Edge	Spatial	Linear	Edge detection (isotropic)	Sum = 0, 2nd derivative
Unsharp Mask	Sharpening	Spatial	Linear	Controlled sharpening	Original + k(Original−Blur)
Sobel	Edge Detection	Spatial	Linear	Directional edges	1st derivative, Gx & Gy
Canny	Edge Detection	Spatial	Multi-stage	Best overall edge detection	Uses Sobel + hysteresis
Low-pass (Freq)	Smoothing	Frequency	Linear	Removing high-freq noise	Circular mask
High-pass (Freq)	Sharpening	Frequency	Linear	Enhancing edges	Inverse of low-pass mask

2. Comparison Table — All Noise Types

Noise Type	Cause	Appearance	Best Removal Filter
Gaussian Noise	Sensor/electronic noise	Fine grainy variation everywhere	Gaussian / Mean filter
Salt-and-Pepper	Transmission errors, faulty sensors	Random black & white pixels	Median filter
Speckle Noise	Coherent imaging (radar, ultrasound)	Granular, multiplicative pattern	Median / Speckle-specific filters
Poisson (Shot) Noise	Photon counting statistics (low light)	Grainy noise in dark images	Averaging / specialized denoising
Periodic Noise	Electrical/mechanical interference	Repetitive stripe/pattern	Frequency domain notch/band-reject filter

---

3. One-Page Cheat Sheet

Filtering = neighborhood-based pixel modification. Two domains: **Spatial** (direct pixel ops) & **Frequency** (via FFT).

Kernel/Mask/Filter ≈ same concept — small matrix defining the operation. Odd size (3x3, 5x5) for a clear center.

Convolution: flips kernel, slides over image, multiplies & sums. `g = f * h`. **Correlation**: same as convolution but **no flipping** (OpenCV's `filter2D` = correlation). **Convolution Theorem**: Convolution (spatial) = Multiplication (frequency).

Smoothing (Low-Pass) Filters

Filter	Kernel Idea
Mean	1/9 × [[1,1,1],[1,1,1],[1,1,1]]
Gaussian	1/16 × [[1,2,1],[2,4,2],[1,2,1]]
Median	Sort neighborhood → pick middle value

Sharpening (High-Pass) Filters

- **Laplacian:** `[[0,1,0],[1,-4,1],[0,1,0]]` → 2nd derivative, sum = 0, isotropic, detects zero-crossings.
- **Unsharp Masking:** `Sharpened = Original + k(Original − Blurred)`

Edge Detection

- **Sobel**: 1st derivative, separate Gx & Gy kernels, gives magnitude + direction. `G = √(Gx² + Gy²)`, `θ = tan⁻¹(Gy/Gx)`
- **Canny (5 steps)**: Gaussian smoothing → Sobel gradient → Non-max suppression → Double thresholding → Hysteresis edge tracking. Best overall edge detector.

Frequency Domain

- Steps: **FFT → apply mask (LPF/HPF) → Inverse FFT.**
- LPF = blur (removes high freq); HPF = sharpen (removes low freq).
- Ideal filter → ringing artifact; Gaussian/Butterworth filter → smoother, less ringing.

Morphological Operations (Binary Images)

- **Erosion**: shrinks white regions — ALL kernel pixels must match (removes noise).
- **Dilation**: grows white regions — ANY kernel pixel match (fills gaps).
- **Opening** = Erosion → Dilation (removes small noise).
- **Closing** = Dilation → Erosion (fills small holes).

Orthogonal Transforms

- **DFT**: complex, general frequency transform.
- **DCT**: real-valued, best energy compaction → used in **JPEG**.
- **Walsh-Hadamard**: fast, only +1/-1 values.
- **Wavelet**: time + frequency localization → used in JPEG2000.

Noise Types Quick Recall

- Gaussian → Mean/Gaussian filter
- Salt-and-Pepper → Median filter
- Speckle → ultrasound/radar images
- Periodic → notch filter (frequency domain)

---

4. Five Commonly Asked University Questions

Q1. What is the difference between convolution and correlation? Convolution flips the kernel 180° before sliding over the image, while correlation does not flip it. For symmetric kernels (e.g., Gaussian), both produce identical results. OpenCV's `filter2D()` performs correlation, not true convolution.

Q2. Why is the median filter better than the mean filter for salt-and-pepper noise? The mean filter averages all neighborhood values, so extreme noise pixels (0 or 255) pull the average away from the true value and blur the image. The median filter instead picks the middle sorted value, so isolated noise spikes are discarded and edges are preserved better.

Q3. Explain the steps of the Canny edge detection algorithm. Canny works in 5 stages: (1) Gaussian smoothing to reduce noise, (2) gradient computation using Sobel operators for magnitude and direction, (3) non-maximum suppression to thin edges, (4) double

thresholding to classify strong/weak/non-edge pixels, and (5) edge tracking by hysteresis, keeping weak edges only if connected to strong ones.

Q4. Differentiate between erosion and dilation with examples. Erosion shrinks white (foreground) regions in a binary image — a pixel stays white only if the entire structuring element fits inside the foreground; it is used to remove small noise. Dilation grows white regions — a pixel becomes white if any part of the structuring element overlaps the foreground; it is used to fill small holes and connect broken parts.

Q5. What is the difference between spatial domain and frequency domain filtering? Spatial domain filtering directly manipulates pixel values using a kernel via convolution and is simple but slower for large kernels. Frequency domain filtering first transforms the image using FFT, applies a filter mask (low-pass/high-pass), and then applies an inverse FFT; it is efficient for large-scale and periodic noise removal but can introduce ringing artifacts.

---

5. Complete OpenCV Program (All Concepts in One)

Demonstrates: Mean, Gaussian, Median, Laplacian, Sobel, Canny, Sharpening (Unsharp Masking), Adding Gaussian noise, Adding Salt-and-Pepper noise, and Noise Removal.

```python
```

```python
import cv2
import numpy as np

# ---------- Load image (grayscale) ----------
img = cv2.imread('image.jpg', 0)

# ---------- 1. Smoothing Filters ----------
mean_filtered     = cv2.blur(img, (5, 5))
gaussian_filtered = cv2.GaussianBlur(img, (5, 5), 0)
median_filtered   = cv2.medianBlur(img, 5)

# ---------- 2. Sharpening ----------
laplacian = cv2.convertScaleAbs(cv2.Laplacian(img, cv2.CV_64F))
blurred   = cv2.GaussianBlur(img, (9, 9), 10)
unsharp   = cv2.addWeighted(img, 1.5, blurred, -0.5, 0)     # Unsharp masking

# ---------- 3. Edge Detection ----------
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel  = cv2.convertScaleAbs(cv2.magnitude(sobelx, sobely))
canny  = cv2.Canny(img, 100, 200)

# ---------- 4. Add Gaussian Noise ----------
gauss_noise  = np.random.normal(0, 25, img.shape).astype(np.float32)
noisy_gauss  = np.clip(img.astype(np.float32) + gauss_noise, 0, 255).astype(np.

# ---------- 5. Add Salt-and-Pepper Noise ----------
noisy_sp = img.copy()
prob = 0.05
rnd  = np.random.rand(*img.shape)
noisy_sp[rnd < prob / 2] = 0          # pepper
noisy_sp[rnd > 1 - prob / 2] = 255    # salt

# ---------- 6. Remove Noise (matched filters) ----------
denoised_gauss = cv2.GaussianBlur(noisy_gauss, (5, 5), 0)   # Gaussian noise ->
denoised_sp    = cv2.medianBlur(noisy_sp, 5)                # Salt-pepper noise

# ---------- Save all results ----------
results = {
    'mean.jpg': mean_filtered, 'gaussian.jpg': gaussian_filtered, 'median.jpg':
    'laplacian.jpg': laplacian, 'unsharp.jpg': unsharp,
    'sobel.jpg': sobel, 'canny.jpg': canny,
    'noisy_gaussian.jpg': noisy_gauss, 'noisy_saltpepper.jpg': noisy_sp,
    'denoised_gaussian.jpg': denoised_gauss, 'denoised_saltpepper.jpg': denoise
}
for name, out in results.items():
```

```
    cv2.imwrite(name, out)

print("All filtering, edge detection, noise addition & denoising operations com
```

Exam tip: Remember the *matching logic* — **Gaussian noise → Gaussian/Mean filter, Salt-and-pepper noise → Median filter.** This pairing is frequently asked in viva/exams.
